Semi-Supervised Isolation Forest for Anomaly Detection

Luca Stradiotti* Lorenzo Perini* Jesse Davis*

Abstract

Anomaly detection algorithms attempt to find instances that deviate from the expected behavior. Because this is often tackled as an unsupervised task, anomaly detection models rely on exploiting intuitions about what constitutes anomalous behavior. These typically take the form of data-driven heuristics that measure the anomalousness of each instance. However, the effectiveness of unsupervised detectors are limited by the validity of their intuition. Because these are not universally true, one can improve the detectors' performance by using a semi-supervised approach that exploits a few labeled instances. This paper proposes a novel semi-supervised tree ensemble based anomaly detection framework. We compare our proposed approach to several baselines and show that it achieves comparable performance to state-of-the-art neural networks on six real-world and 14 benchmark datasets.

1 Introduction

Anomaly detection is the task of identifying instances that do not conform to the expected normal behavior [3]. Usually, anomalies represent critical events such as unauthorized accesses in computer networks [13], credit card stealing [2], failures in manufacturing [41] or breakdowns in wind turbines [37, 23]. These critical events are associated with privacy issues (e.g., intrusion detection), high monetary costs (e.g., manufacturing failures), or waste of resources (e.g., shutdowns of wind turbines). Therefore, the timely detection of anomalies is an important problem in practice.

Anomaly detection is often tackled from an unsupervised perspective because labels, especially for the anomalies, are usually hard to come by due to their inherent rarity and the cost of obtaining labels [24, 25, 19, 26]. Therefore, unsupervised anomaly detectors are designed based on our intuitive definitions of what constitutes an anomaly, which is typically that they significantly differ from normal instances. These intuitions are operationalized using data-driven heuristics that measure the anomalousness of each instance with respect to the data. Isolation Forest (IF) [16] is a popular method that assumes that anomalies are easier to isolate from all the remaining data when a tree is constructed in a random manner: that is the attribute-value test in each node is selected by randomly picking a feature and its value to split on. This intuitive approach has inspired various extensions, including adapting IF for functional data [32], feature rotation [10, 15], instance clustering in leaf nodes [12], and redesigning the split function [33].

Unfortunately, real-world scenarios often violate the assumptions of unsupervised detectors, impacting their performance. This has led to the development of semi-supervised anomaly detection algorithms [29, 22, 35], which utilize labeled instances to improve detection in cases where unsupervised methods fail.

In this paper, we propose **SSIF** (Semi-Supervised Isolation Forest). In contrast to existing semi-supervised tree-based anomaly detectors [18, 30], we propose a method that exploits both labeled and unlabeled data when learning the trees. A key challenge in this regard is how to effectively blend the information contained in labeled and unlabeled instances. Our method addresses this challenge by taking a probabilistic perspective and constructing informative distributions that bias the selection of a feature and its value to split on towards (1) splits that are more likely to quickly isolate anomalies and (2) features that are likely to contain good splits. Finally, we propose a mechanism for computing an instance's anomaly score that weights paths of the trees based on the labeled instances contained in the leaves. We evaluate our approach on six real-world and 14 benchmark datasets and show that it achieves performance that is better or comparable to the state-of-the-art network-based models.

2 Preliminaries

Let $\mathcal{X}$ be a M dimensional input space and $D = \{x_1, \ldots, x_n\}$ be a training set, where each $x_i \in \mathcal{X}$. We indicate the k-th feature as X_k, for $k \leq M$. Moreover, we assume that some labels are given, such that $D = D_L \cup D_U$, where the instances in $D_L = \{x_1, \ldots, x_l\}$ have labels $Y = \{y_1, \ldots, y_l\}$ with the anomaly class being positive ($y = +1$) and the negative being normal ($y = -1$), while $D_U = \{x_{l+1}, \ldots, x_n\}$ has no labels. The goal of anomaly detection is to build a detector

*Department of Computer Science, KU Leuven, Belgium, {luca.stradiotti,lorenzo.perini,jesse.davis}@kuleuven.be

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

$f\colon \mathcal{X} \to \mathbb{R}$ that maps examples to anomaly scores, representing the examples' degree of anomalousness.

Isolation Forest (IF) [16] is the most common tree-based unsupervised anomaly detector, and assigns anomaly scores following the intuition that anomalies differ from normals on some features. Thus, anomalies require fewer splits to be isolated (i.e., separated from the other instances) when building an isolation tree. An isolation tree (iTree) comprises nodes T that are either leaves (with no children) or internal (with a test and two child nodes, $T_<$ and $T_\geq$). The test is a pair (X_k, s) where X_k is the (selected) k-th feature and s is a split value, such that the inequality $X_k < s$ partitions the feature space into $T_<$ and $T_\geq$. To build an iTree, two random variables, $\mathcal{K}$ and $\mathcal{S}$, are used iteratively to randomly (i.e., uniformly) select features X_k and sample split values s until the tree reaches a predetermined height or a leaf contains only one instance. Because anomalies require fewer splits to be isolated, IF assigns anomaly scores based on the path length h through iTrees: $f_{IF}(x) = 2^{-\frac{\sum_{j=1}^{T} h_j(x)}{T \cdot c(n)}}$, where T is the number of iTrees in the ensemble, and $c(n)$ is the average path length of unsuccessful search in a Binary Search Trees [27].

3 SSIF: Semi-Supervised Isolation Forest

Developing a semi-supervised version of IF poses two important challenges. First, because of the limited number of labels, it is necessary to combine the information contained in both labeled and unlabeled data. Second, we have to use the labeled information in a way that aligns with the underlying heuristic employed by IF of trying to isolate anomalies as quickly as possible.

To tackle these challenges, we propose **SSIF** (Semi-Supervised Isolation Forest). At a high-level, our approach differs from a traditional IF in two important ways. First, instead of picking the attribute to split on and its split value uniformly at random, we design a probabilistic semi-supervised approach that measures the informativeness of potential splits, while adhering to the goal of isolating anomalies with as few splits as possible. Second, it uses an anomaly score function that leverages the training labels in the leaves.

3.1 Tree Induction.

Our goal is to induce a tree that leverages labeled and unlabeled instances to isolate the anomalies with as few splits as possible. This requires designing a split criteria that favors the selection of splits likely to isolate (potential) anomalies. In line with IF, we keep a probabilistic setting and design (1) a split distribution function such that splits with higher quality have higher probability, and (2) a feature selection function such that features with higher quality splits are more likely to be picked.

3.1.1 Split Distribution.

Given a feature X_k, the split distribution needs to measure the quality of any split $X_k < s$. Because a node can have both labeled and unlabeled instances, our function for evaluating the quality of a split has a labeled and unlabeled component. The labeled component focuses on isolating labeled anomalies while keeping together labeled normals. The unlabeled component aims to effectively separate small clusters of unlabeled instances (potential anomalies) from larger ones (potential normals). Both components estimate split quality by measuring the weighted average reduction of uncertainty in the child nodes: the labeled component uses class conditional entropy to capture prediction uncertainty, and the unlabeled component employs variance to capture data uncertainty. Finally, we derive the split distribution by combining these two components.

Labeled component. Within a specific node, there may not be labeled examples from both classes available. This is particularly true for deeper internal nodes because the tree builder tries to isolate the anomalies as soon as possible. Therefore, we distinguish between three mutually exclusive scenarios:

a) Both class labels are available. In this case, we evaluate the quality of a split only using the labeled examples. Because entropy is a metric of class impurity (i.e., high entropy indicates mixed class labels), we measure the quality of a split s on the feature X_k for the node T using the standard information gain metric [28]

(3.1)
$$\mathcal{L}_T(s) = \mathcal{H}\left(Y|X_k^T\right) - \left[w_< \mathcal{H}\left(Y|X_k^{T_<}\right) + w_\geq \mathcal{H}\left(Y|X_k^{T_\geq}\right)\right]$$

where $\mathcal{H}(Y|X) = -\sum_{x\in\mathcal{X}, y\in\mathcal{Y}} p(x,y) \log\left[p(x,y)p(x)^{-1}\right]$ is the class conditional entropy, $X_k^{T_<}$ and $X_k^{T_\geq}$ are the instances on the left ($T_<$) and right ($T_\geq$) child nodes, with $w_<$ and $w_\geq$ being their size proportions.

b) Only anomaly labels are available. Because anomalies are by definition rare, most of the unlabeled instances are normals. Therefore, when a node only has labeled anomalies we assume that all unlabeled instances are normal and use Equation 3.1 to evaluate the quality of each split.

c) Only normal labels are available. Because our task is to isolate anomalies with few splits, we favor splits that sort the labeled normals to same child node. Conceptually, our key insight is that anomalous instances are more likely located outside of the interval that goes from the lowest ($\underline{x}$) to the highest ($\overline{x}$) value of the labeled normal instances. Thus, we measure the quality

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

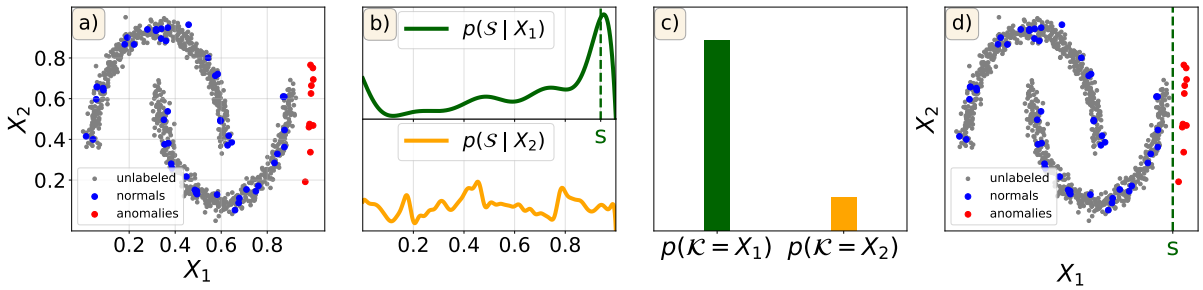


Figure 1: Illustration of the SSIF's steps on a 2D toy dataset (Fig. a): we 1) compute the split distributions for the two features X_1 and X_2(Fig. b), 2) compute the feature selection distribution (Fig. c), 3) sample the feature X_1 from the feature selection distribution and finally we sample s as the following split value (Fig. d).

of a split outside $[\underline{x}, \overline{x}]$ as its distance to the edges

$$\mathcal{L}_T(s) = \begin{cases} \min\{|s-\underline{x}|, |s-\overline{x}|\} & \text{if } s \notin [\underline{x}, \overline{x}]\,; \\ 0 & \text{otherwise} \end{cases}.$$

Unlabeled component. In the absence of labels, we rely on data-driven intuitions to measure the split's quality. Ideally, we want to partition the data into two groups such that instances in the same group are relatively close whereas instances in opposite groups are relatively far. Thus, we employ variance as an uncertainty metric on the features and measure the intra-class variance of the two children's sets of instances (left $<$, right $\geq$). However, this would naturally tend to favor splits close to the median and disadvantage splits at the edges of the interval. Therefore, following [14], we employ an additive term that accounts for imbalanced groups of instances, and design the unlabeled component as

$$\mathcal{U}_T(s) = \overbrace{\mathbb{P}(X_k^T<s)\log\mathbb{P}(X_k^T<s)+\mathbb{P}(X_k^T\geq s)\log\mathbb{P}(X_k^T\geq s)}^{\text{Additive term to account for imbalanced groups}} - 0.5\underbrace{\log\left[\mathbb{P}(X_k^T<s)\cdot\sigma_<^2+\mathbb{P}(X_k^T\geq s)\cdot\sigma_\geq^2\right]}_{\text{Data variance}},$$

where $\mathbb{P}(X_k^T < s)$ is the proportion of instances $< s$.

Deriving the split distribution. We must weigh the two components differently when combining them because (1) labels are scarce and should be maximally leveraged, and (2) unlabeled instances are less indicative of the split quality. We set the weights to the proportion of labeled instances c_T at node T and normalize the sum to force the density to have integral equal to 1:

$$p(\mathcal{S}=s|X_k) = \frac{(1+c_T)\mathcal{L}_T(s)+(1-c_T)\mathcal{U}_T(s)}{\int_{\mathcal{A}_k}^{\mathcal{B}_k}\left[(1+c_T)\mathcal{L}_T(v)+(1-c_T)\mathcal{U}_T(v)\right]dv}$$

where $\mathcal{A}_k$, $\mathcal{B}_k$ are, respectively, the min and max of X_k, and $\mathcal{L}_T$, $\mathcal{U}_T$ are normalized to be on the same scale. Note that the derived split distribution $p(\mathcal{S}=s|X_k)$ can naturally deal with discrete features, as it is constant in regions without examples.

3.1.2 Feature-Selection Distribution. Intuitively, if all possible splits for a feature have the same quality, we would not care which one we picked and the split distribution would be uniform. However, because usually some splits are more effective at isolating anomalies, the feature selection distribution must identify the (informative) feature, i.e., the ones that have high-quality splits. Consequently, a feature's informativeness is tied to how much a feature's split distribution deviates from the uniform distribution (i.e., the more peaked it is) with higher deviations indicating more informative features. Therefore, we measure feature informativeness as the dissimilarity between its split distribution $p(\mathcal{S}|X_k)$ and a uniform distribution $p(V)$ according to the Kullback-Leibler (KL) divergence [17]

$$p(\mathcal{K}=k) = \frac{\mathrm{KL}\left(\mathcal{S}|X_k \| V\right)}{\sum_{z\leq M}\mathrm{KL}\left(\mathcal{S}|X_z \| V\right)}.$$

where results are normalized to obtain a proper discrete density. We motivate our choice further in the online supplement by demonstrating that such formulation assigns higher probabilities to steeper (i.e., more informative) split distributions.

3.2 Leveraging Training Labels to Compute Anomaly Scores. Because of its probabilistic nature, SSIF may select unlikely samples, which can degrade performance. For instance, it may incorrectly isolate labeled normals due to unlikely splits or select non-informative features meaning that more splits are needed to isolate anomalies which will increase their path lengths. Therefore, instead of simply assigning an anomaly score to instance x by considering its path5 length within an ensemble of trees, our method SSIF

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

adjusts the anomaly score based on labeled data. We consider all leaves where a training instance $\tilde{x}$ with label $\tilde{y}$ is isolated. If a test instance x falls through a tree into a leaf node that contains $\tilde{x}$, it is likely to share the same label $\tilde{y}$ and therefore we reduce (or increase) the path length for that tree. Specifically, we fix for each tree t the standard path length h by employing $h_t^*(x) = h_t(x)[1 - \tilde{y}]$. If the leaf contains an unlabeled instance ($\tilde{y} = 0$), the path remains unchanged.

Finally, similarly to IF, SSIF assigns the anomaly score to an instance x as $f(x) = 2^{-\frac{\sum_{j=1}^{T} h_j^*(x)}{T \cdot c(n)}}$

3.3 SSIF implementation: technical details. Similarly to most of the existing tree-based models, SSIF introduces four hyperparameters. To reduce the computational cost, we construct each tree (1) using a proportion $\psi \in [0, 1]$ of the training set's instances (uniformly drawn, no replacement), and (2) randomly selecting $\bar{M} \leq M$ features to use for the feature selection. To reduce the bias introduced by each tree, we (3) use an ensemble of τ trees and (4) end the tree construction when a height limit λ is reached.

Methodology summary. Overall, the key step while learning a tree top-down is selecting an attribute-value pair to split on. We design the feature and split distributions and independently sample from each of them. Algorithm 1 outlines how to build each tree. Then, making a prediction is done separately from learning (see Section 3.2).

Algorithm 1 BuildTree(D_U, D_L, Y, T, e, λ, $\bar{M}$)

Inputs: D_U - unlabeled data, D_L - labeled data, Y - labels, T - current tree node, e - current tree height, λ - height limit, $\bar{M}$ - reduced number of features.

```
if end condition then
  return T_leaf{D_U,D_L,Y,T}
end if
draw M̄ features to get X̄
for k in X̄ do
  compute p(S | X_k) as in Section 3.1.1
end for
compute p(K) as in Section 3.1.2
sample k from p(K) and s from p(S | X_k)
T_< = partition T based on X_k < s
T_≥ = partition T based on X_k ≥ s
return T_internal{D_U,D_L,Y,T,
        left = BuildTree(D_U,D_L,Y,T_<,e+1,λ,M̄),
        right = BuildTree(D_U,D_L,Y,T_≥,e+1,λ,M̄),
        SplitAtt = X_k,
        SplitValue = s}
```

4 Related Work

Semi-supervised anomaly detectors can be distinguished into four categories based on how they use labels: incomplete supervision-based, propagation-based, loss-based, and tree-based models [21, 11, 36].

1. Incomplete supervision-based semi-supervised anomaly detectors assume all training instances to be normal and learn their distribution to flag out-of-distribution instances as anomalies [4, 40]. For example, LUNAR [7] uses a Graph Neural Network to detect anomalies through a message-passing algorithm. These models can be adapted to use anomaly labels by removing all the labeled anomalies from the training set.

2. Propagation-based semi-supervised anomaly detectors assume that similar/close instances share the same label. Thus, a test instance's prediction depends on how similar/close it is to the given training labeled instances [5]. For example, SSDO [35] and SSkNNo [34] use an unsupervised anomaly detector to initially assign the anomaly score to an instance, and then they update such a score by computing a distance-based correction based on the neighbors' labels.

3. Loss-based semi-supervised anomaly detectors build an ad-hoc loss function that includes both labeled and unlabeled instances to distinguish between normals and anomalies [39, 6]. For example, SSAD [8] and DeepSAD [29] learn a hypersphere that encapsulates the unlabeled and normally labeled training instances while leaving out the anomalies. Alternatively, Repen [20] uses triplet networks to learn expressive feature representations of the instances, while DevNet [22] leverages a limited number of labeled anomalies to perform end-to-end anomaly score learning.

4. Tree-based semi-supervised anomaly detectors learn the patterns of normal (and potentially anomalous) instances by creating a sequence of splits that isolate the anomalous instances from the normals. The key assumption is that there exist some features where anomalies can be easily isolated by a split. For example, Hif [18] uses IF to randomly split the instance space, and then it leverages the labeled anomalies to measure the proximity to anomalous instances. Alternatively, Siddiqui et al. [30] propose to exploit the training labels to assign different weights to the IF's trees, where trees with a more discriminative ranking get higher weights.

5 Experiments

We experimentally answer the following questions[1]:

[1] Additionally, the online supplement evaluates how SSIF's sampling technique compares to a simple max-value selection and whether SSIF is able to cope with data distributions where IF is known to struggle.

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

Q1: How does SSIF compare to existing methods?

Q2: What is the effect of SSIF's components on its final performance?

Q3: What is the impact of SSIF's hyperparameters?

5.1 Experimental setup

Methods. We run an extensive analysis by comparing our method SSIF[2] against seven existing state-of-the-art semi-supervised methods: 1) LUNAR [7] as an incomplete supervision-based model, 2) SSDO [35] as a propagation-based model, 3) SSAD [8], DeepSAD [29], DevNet [22] and REPEN [20] as loss-based models, 4) HIF [18] as a tree-based model. Moreover, for the sake of completeness, we include IF [16] as an unsupervised baseline. All these methods are described in detail in Section 4.

Data. We run our experimental analysis on six real-world wind turbine datasets with two different tasks, and 14 benchmark datasets. For two turbine datasets (T15 and T21), the task is to detect ice formation (i.e., the anomalies) on their blades, which could potentially damage the turbines and slow power production [38]. Various measurements (e.g., wind speed and power) are collected every seven seconds for either two months (T15) or one month (T21). Following [38], we construct feature-vectors by averaging over segments of 10 minutes. For the other four turbine datasets (T01, T06, T07, T11), the goal is to detect component failures. Their SCADA signals are collected every 10 minutes for two years. We complement our analysis with 14 benchmark datasets widely used in anomaly detection [9]. Table 1 in the online supplement shows their characteristics.

Setup. For each of the 20 datasets, we employ the following procedure: (i) we split the dataset into training and test set (80%−20%), (ii) we simulate gradually increasing the percentage of labels $c = \frac{l}{n}$ from 5% to 50% (every 5%), (iii) we train the models using the unlabeled and the labeled instances, (iv) we use the learned methods to assign anomaly scores to the test instances, and (v) we measure their performance using the Area Under the ROC curve (AUROC), as it is a standard evaluation metric in anomaly detection [1]. With the goal of obtaining robust results and reducing random effects, we run steps (ii) to (v) 10 times and report averages and standard deviations. The combination of 20 datasets, 10 repetitions, and 10 label percentages results in a total of 2000 experiments. All experiments were run on an Intel(R) Xeon(R) Silver 4214 CPU with 128GiB of memory. The experiments finished in $\sim$ 48 hours.

Hyperparameters. SSIF has four hyperparameters (see Section 3.3). We set the number of instances $\psi = \max(\frac{n}{3}, 128)$, the height limit $\lambda = 2\lceil(log_2\psi)\rceil$, the ensemble size $\tau = 100$, and the number of features $\bar{M} = \frac{M}{5}$. All these hyperparameters are carefully analyzed in Q3.

We set the baselines' hyperparameters to the values proposed by the original papers (default values) [31]. SSDO uses $k = 30$, $\alpha = 2.3$, and IF as unsupervised prior. SSAD sets $k, \eta_u, \eta_l = 1$ and uses a Gaussian kernel. HIF employs $\tau = 100$, $\psi = 256$, $\lambda = 1.2\lceil(log_2\psi)\rceil$, and $\alpha_1, \alpha_2 = 0.5$, while IF has $\tau = 100$, $\psi = 256$, $\lambda = \lceil(log_2\psi)\rceil$. The four network-based baselines rely on the structure provided in the paper. In addition, DeepSAD uses $\eta = 1$, 50 epochs for training with batch size = 64, and Adam as optimizer with learning rate = 0.001. DevNet uses 50 epochs with 20 mini-batches (per epoch), and RMSProp as optimizer with learning rate = 0.001. REPEN uses $M = 20$, 30 epochs with batch size = 64, and Adadelta as optimizer with learning rate = 0.001. Finally, LUNAR uses $k = 5$, 200 epochs, and Adam as optimizer with a learning rate of 0.001.

5.2 Experimental results

Q1: Methods comparison. Figure 2 (left) shows each method's average AUROC aggregated over all 10 iterations and 20 datasets for each of the 10 label percentages. SSIF consistently obtains the highest (best) average AUROC. Even with limited labels ($c <$ 10%) SSIF performs better than the existing tree-based approaches HIF and IF, and the performance gap grows as the label percentage c increases. With just 15% of labeled data, SSIF's average AUROC surpasses the nearest runners-up by more than 10%. When compared to IF, SSIF's AUROC yields an improvement of around 9% for low c ($\leq$ 20%), and more than 16% for high c ($\geq$ 40%). Averaging over the labeled percentages, SSIF improves the performance by more than 10% vs HIF, SSDO and DeepSAD, 15% vs DevNet, 25% LUNAR and REPEN and 48% vs SSAD.

Figure 3 shows the AUROC as a function of label percentage c for three representative datasets: Shuttle, Spambase, and Cardiotocography. SSIF consistently outperforms all the baselines on Shuttle and SpamBase across all label percentages. In some cases, like for Cardiotocography, SSIF obtains lower performance for lower label percentages ($c < 20\%$), but it becomes competitive when acquiring more labels ($c \geq 20\%$). Importantly, SSIF performs more consistently across datasets than the other baselines. For instance, SSDO and De-

[2] https://github.com/stralu9/Semi_Supervised_Isolation_Forest

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

vNet perform well on Cardiotocography, while they are among the worst in Shuttle. Moreover, SSIF's performance steadily increases as more labeled instances are provided while the competitors have wider oscillations. Section 2 in the online supplement provides the detailed results for all the datasets.[3]

Next, for each label percentage we compute the average rank across all datasets according to their AUROC values. Rank 1 indicates the highest (best) AUROC while rank 9 the lowest (worst). Figure 2 (right) shows the average rankings per label percentage. Overall, SSIF obtains always the lowest (best) average rank. In addition, it achieves the best rank on 52% of the datasets, while the best competitors SSDO and DeepSAD do so in only 16% of the experiments. SSIF's rank becomes better when gradually increasing the proportion of labels, starting from 2.45 (for $c = 5\%$) and ending at 1.77 (for $c = 50\%$). Then, we compare SSIF in a 1vs1 fashion against the most competitive baselines. Over all 2000 considered experiments, SSIF outperforms its two nearest competitors DeepSAD and DevNet, on 69% and 70% of the experiments respectively.

Q2: Ablation study. To measure the impact of each component of SSIF, we remove them, one at a time, in a hierarchical fashion before training and testing the remaining components of the model. We use a hierarchical approach because some components cannot be used without others (e.g., the feature selection needs the split distribution). Because the removal order matters, we sort the components by importance and obtain a total of 4 simplified versions of SSIF: (i) $\mathcal{L}$+$\mathcal{U}$+$\mathcal{K}$ drops our modification for the path length and uses $h^* = h$, (ii) $\mathcal{L}$+$\mathcal{U}$ further drops our feature selection and selects it uniformly, (iii) $\mathcal{L}$ further drops the unlabeled component and relies only on the labeled component. Finally, for the sake of completeness, we consider also (iv) $\mathcal{U}$+$\mathcal{K}$+h^* that removes only the labeled component from the full model.

Figure 4 shows the comparison between our full model SSIF and its 4 simplified versions on 5 representative datasets. All the components have a significant contribution to the final performance: (i) removing the path length modification ($\mathcal{L}$+$\mathcal{U}$+$\mathcal{K}$) yields an average drop in performance of 3.9%, (ii) dropping the feature selection ($\mathcal{L}$+$\mathcal{U}$) further decreases the performance by 8.2%, and, finally, (iii) leaving only the labeled component make the performance fall by 1%. While (iv) only removing the labeled component would result in a performance drop of 2.1%.

[3] https://github.com/stralu9/Semi_Supervised_Isolation_Forest/blob/main/Supplement.pdf

Finally, we compare IF to our method when no labels are provided. In this setting, SSIF still samples an informative split but it only considers the unlabeled component explained in Section 3.1.1. This variant of SSIF outperforms IF on 11 datasets while IF has a higher average AUROC on 5 datasets. The detailed results can be found in Table 2 (supplement).

Q3: Hyperparameter sensitivity analysis. We measure the sensitivity of SSIF to the values of its four hyperparameters. We vary the value of each hyperparameter in turn while keeping the others fixed. We consider the following ranges: the number of samples used to build each tree $\psi \in \{\frac{1}{10}, \frac{1}{5}, \frac{1}{3}, \frac{1}{2}, 1\} \cdot n$, the reduced number of features $\bar{M} \in \{\frac{1}{20}, \frac{1}{10}, \frac{1}{5}, \frac{1}{2}, 1\} \cdot M$, the number of trees in the ensemble $\tau \in \{20, 50, 100, 200, 500\}$, and the height limit for the stopping criteria $\lambda \in \{0.5, 1, 2, 3, 4\} \cdot \lceil (log_2 \psi) \rceil$. Figure 5 shows the results. Overall, SSIF's performance is quite stable for most of the considered hyperparameter values. Neither $\bar{M}$ nor τ have a large impact on the final performance: on average, SSIF's AUROCs with alternative hyperparameter values do not differ by more than 2% from its AUROC with default hyperparameters. This happens because (1) $\bar{M}$ always includes the most informative features, and (2) the ensemble size enables enough diversity in the forest. On the other hand, ψ and λ can have a larger impact on SSIF's performance for some datasets. For these hyperparameters, selecting the lowest value in the considered range ($\lambda = 0.5\lceil (log_2 \psi) \rceil$ and $\psi = \frac{n}{10}$) can degrade performance.

6 Conclusion

Anomaly detection is of great practical importance because it involves identifying critical events. Unsupervised anomaly detectors have achieved good results by using data-driven heuristics that model the assumption that anomalies differ significantly from normals. However, their assumptions are often violated in real-world applications, which may adversely affect their performance. We proposed SSIF (Semi-Supervised Isolation Forest), a probabilistic model that leverages both labeled and unlabeled data when learning the trees. We addressed the key challenge of blending the information contained in labeled and unlabeled instances by constructing an informative split distribution and proposed a mechanism for computing an example's anomaly score that depends on the labeled instances contained in the tree leaves. Our experimental results on 14 benchmark and six real-world datasets show that SSIF achieves performance that is comparable to the state-of-the-art neural-network-based models, demonstrating the potential of our proposed approach in real-world applications.

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

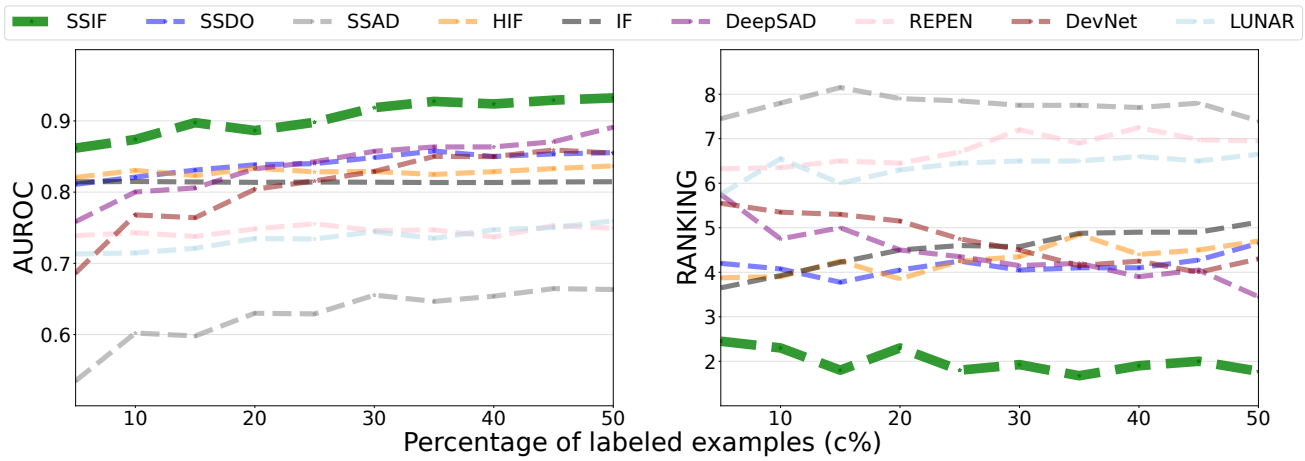


Figure 2: (left) Average AUROC and (right) average rank for each method and percentage of labels $c = \frac{l}{n}$ aggregated over all iterations and datasets. For each value of c, SSIF obtains both the highest (best) average AUROC and the lowest (best) average ranking position.

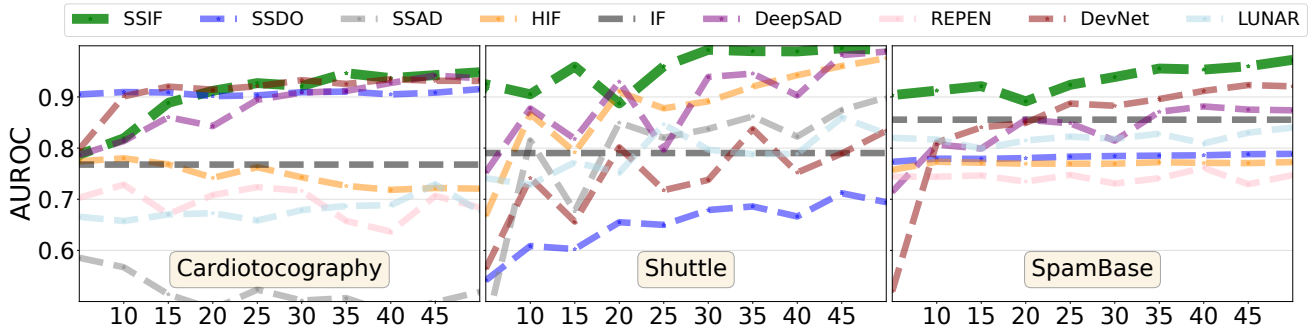


Figure 3: Average test set AUROC for each method and percentage of labels $c = \frac{l}{n}$ aggregated over all iterations on three representative datasets. SSIF outperforms all the other competitors in Shuttle and SpamBase and is competitive in Cardiotocography starting from $c = 20\%$.

Acknowledgements

This research is supported by the Flemish Government under the "Onderzoeksprogramma Artificiële Intelligentie (AI) Vlaanderen" programme [LS,LP,JD], an FB Ph.D. fellowship by FWO-Vlaanderen (grant 1166222N) [LP], and KUL Research Fund iBOF/21/075 [JD].

References

[1] G. O. Campos, A. Zimek, J. Sander, R. J. G. B. Campello, B. Micenková, E. Schubert, I. Assent, and M. E. Houle, *On the evaluation of unsupervised outlier detection: measures, datasets, and an empirical study*, Data Mining and Knowledge Discovery, 30 (2016), pp. 891–927.

[2] P. Chan, W. Fan, A. L. Prodromidis, and S. J. Stolfo, *Distributed data mining in credit card fraud detection*, IEEE Intelligent Systems, 14 (1999), pp. 67–74.

[3] V. Chandola, A. Banerjee, and V. Kumar, *Anomaly detection: A survey*, ACM Comput. Surv., 41 (2009), pp. 15:1–15:58.

[4] Z. Chen, C. K. Yeo, B. Lee, and C. T. Lau, *Autoencoder-based network anomaly detection*, in WTS, IEEE, 2018, pp. 1–5.

[5] J. Deng and E. T. Brown, *SSDBCODI: semi-supervised density-based clustering with outliers detection integrated*, CoRR, abs/2208.05561 (2022).

[6] F. Gao, J. Li, R. Cheng, Y. Zhou, and Y. Ye, *ConNet: Deep semi-supervised anomaly detection based on sparse positive samples*, IEEE Access, 9 (2021), pp. 67249–67258.

[7] A. Goodge, B. Hooi, S. Ng, and W. S. Ng, *LUNAR: unifying local outlier detection methods via graph neural networks*, in AAAI, AAAI Press, 2022, pp. 6737–6745.

[8] N. Görnitz, M. Kloft, K. Rieck, and U. Brefeld, *Toward supervised anomaly detection*, J.

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

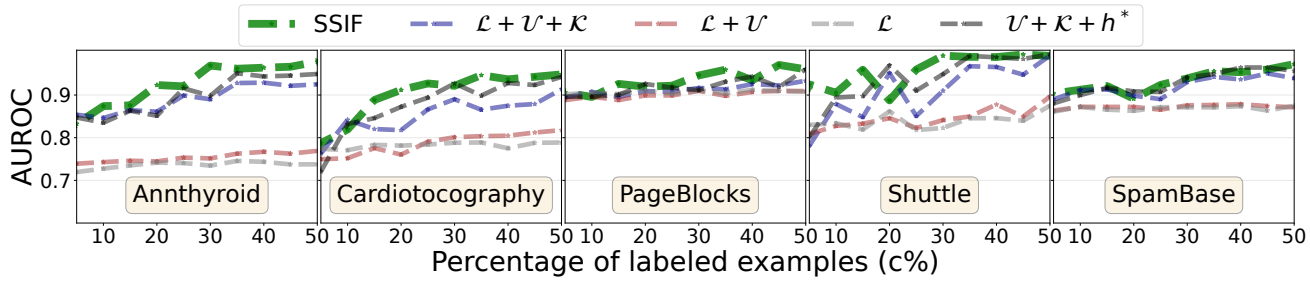


Figure 4: The impact of SSIF components' contribution on the average test set AUROC as a function of the percentage of labels provided ($c\%$). All the model components have a significant contribution to the final performance.

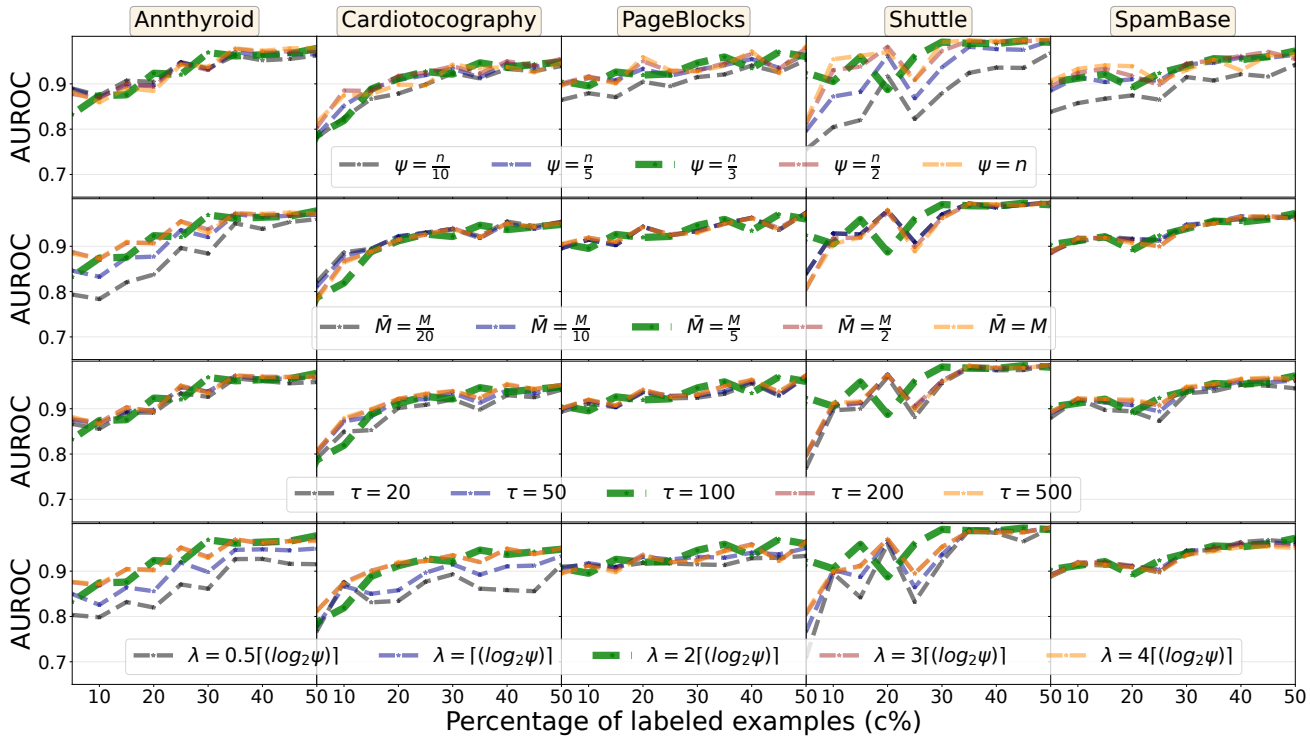


Figure 5: Impact of SSIF's hyperparameters (a) ψ, (b) $\bar{M}$, (c) τ and (d) λ on the average AUROC as a function of the percentage of labels ($c\%$). Overall, SSIF is robust to different choices of values for $\bar{M}$ and τ while its performance deteriorates when decreasing ψ and λ

Artif. Intell. Res., 46 (2013), pp. 235–262.

[9] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, *ADBench: Anomaly detection benchmark*, in NeurIPS, 2022.

[10] S. Hariri, M. C. Kind, and R. J. Brunner, *Extended Isolation Forest*, IEEE Trans. Knowl. Data Eng., 33 (2021), pp. 1479–1489.

[11] M. Jiang, C. Hou, A. Zheng, X. Hu, S. Han, H. Huang, X. He, P. S. Yu, and Y. Zhao, *Weakly supervised anomaly detection: A survey*, CoRR, abs/2302.04549 (2023).

[12] P. Karczmarek, A. Kiersztyn, W. Pedrycz, and E. Al, *K-Means-based isolation forest*, Knowl. Based Syst., 195 (2020), p. 105659.

[13] V. Kumar, *Parallel and distributed computing for cybersecurity*, IEEE Distributed Syst. Online, 6 (2005).

[14] T. Kurita, N. Otsu, and N. N. Abdelmalek, *Maximum likelihood thresholding based on population mixture models*, Pattern Recognit., 25 (1992), pp. 1231–1240.

[15] J. Lesouple, C. Baudoin, M. Spigai, and J. Tourneret, *Generalized isolation forest for anomaly detection*, Pattern Recognit. Lett., 149 (2021), pp. 109–119.

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited

[16] F. T. Liu, K. M. Ting, and Z. Zhou, *Isolation forest*, in ICDM, IEEE Computer Society, 2008, pp. 413–422.

[17] D. J. C. MacKay, *Information theory, inference, and learning algorithms*, Cambridge University Press, 2003.

[18] P. Marteau, S. Soheily-Khah, and N. Béchet, *Hybrid isolation forest - application to intrusion detection*, CoRR, abs/1705.03800 (2017).

[19] R. A. Maxion and K. M. C. Tan, *Benchmarking anomaly-based detection systems*, in DSN, IEEE Computer Society, 2000, pp. 623–630.

[20] G. Pang, L. Cao, L. Chen, and H. Liu, *Learning representations of ultrahigh-dimensional data for random distance-based outlier detection*, in KDD, ACM, 2018, pp. 2041–2050.

[21] G. Pang, C. Shen, H. Jin, and A. van den Hengel, *Deep weakly-supervised anomaly detection*, in Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023, pp. 1795–1807.

[22] G. Pang, C. Shen, and A. van den Hengel, *Deep anomaly detection with deviation networks*, in KDD, ACM, 2019, pp. 353–362.

[23] L. Perini, P. Bürkner, and A. Klami, *Estimating the contamination factor's distribution in unsupervised anomaly detection*, in ICML, vol. 202 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 27668–27679.

[24] L. Perini, V. Vercruyssen, and J. Davis, *Quantifying the confidence of anomaly detectors in their example-wise predictions*, in ECML/PKDD (3), vol. 12459 of Lecture Notes in Computer Science, Springer, 2020, pp. 227–243.

[25] L. Perini, V. Vercruyssen, and J. Davis, *Transferring the contamination factor between anomaly detection domains by shape similarity*, in AAAI, AAAI Press, 2022, pp. 4128–4136.

[26] L. Perini, V. Vercruyssen, and J. Davis, *Learning from positive and unlabeled multi-instance bags in anomaly detection*, in KDD, ACM, 2023, pp. 1897–1906.

[27] B. R. Preiss, *Data Structure and Algorithms: With Object-oriented Design Patterns in Java*, John Wiley & Sons, 1999.

[28] J. R. Quinlan, *Learning decision tree classifiers*, ACM Comput. Surv., 28 (1996), pp. 71–72.

[29] L. Ruff, R. A. Vandermeulen, N. Görnitz, A. Binder, E. Müller, K. Müller, and M. Kloft, *Deep semi-supervised anomaly detection*, CoRR, abs/1906.02694 (2019).

[30] M. A. Siddiqui, A. Fern, T. G. Dietterich, R. Wright, A. Theriault, and D. W. Archer, *Feedback-guided anomaly discovery via online optimization*, in KDD, ACM, 2018, pp. 2200–2209.

[31] J. Soenen, E. Van Wolputte, L. Perini, V. Vercruyssen, W. Meert, J. Davis, and H. Blockeel, *The effect of hyperparameter tuning on the comparative evaluation of unsupervised anomaly detection methods*, in Proceedings of the KDD'21 Workshop on Outlier Detection and Description, Outlier Detection and Description Organising Committee, 2021, pp. 1–9.

[32] G. Staerman, P. Mozharovskyi, S. Clémençon, and F. d'Alché-Buc, *Functional isolation forest*, in ACML, vol. 101 of Proceedings of Machine Learning Research, PMLR, 2019, pp. 332–347.

[33] M. Tokovarov and P. Karczmarek, *A probabilistic generalization of isolation forest*, Inf. Sci., 584 (2022), pp. 433–449.

[34] V. Vercruyssen, W. Meert, and J. Davis, *Transfer learning for anomaly detection through localized and unsupervised instance selection*, in AAAI, AAAI Press, 2020, pp. 6054–6061.

[35] V. Vercruyssen, W. Meert, G. Verbruggen, K. Maes, R. Baumer, and J. Davis, *Semi-supervised anomaly detection with an application to water analytics*, in ICDM, IEEE Computer Society, 2018, pp. 527–536.

[36] M. E. Villa-Pérez, M. Á. Á. Carmona, O. Loyola-González, M. A. Medina-Pérez, J. C. Velazco-Rossell, and K. R. Choo, *Semi-supervised anomaly detection algorithms: A comparative summary and future research directions*, Knowl. Based Syst., 218 (2021), p. 106878.

[37] A. Zaher, S. McArthur, D. Infield, and Y. Patel, *Online wind turbine fault detection through automated scada data analysis*, Wind Energy, 12 (2009), pp. 574 – 593.

[38] L. Zhang, K. Liu, Y. Wang, and Z. B. Omariba, *Ice detection model of wind turbine blades based on random forest classifier*, Energies, 11 (2018), p. 2548.

[39] Y. Zhou, X. Song, Y. Zhang, F. Liu, C. Zhu, and L. Liu, *Feature encoding with autoencoders for weakly supervised anomaly detection*, IEEE Trans. Neural Networks Learn. Syst., 33 (2022), pp. 2454–2465.

[40] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, *Deep autoencoding gaussian mixture model for unsupervised anomaly detection*, in ICLR (Poster), OpenReview.net, 2018.

[41] K. Zope, K. Singh, S. Nistala, A. Basak, P. Rathore, and V. Runkana, *Anomaly detection and diagnosis in manufacturing systems: A comparative study of statistical, machine learning and deep learning techniques*, in Annu. Conf. PHM Soc, vol. 11, 2019.

Copyright © 2024 by SIAM
Unauthorized reproduction of this article is prohibited